German University in Cairo
Department of Computer Science
Dr. Yomna Hassan
Dr. Mary Guindy

Concepts of Programming Languages, Spring Term 2026
Project - Task 2: Haskell

Due: 15^{th} *May 2025*

1. Project Description

In this project, you are going to implement some utilities for a restaurant that help it manage expenses and supplies.

2. Data Types

```
data Month = Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec
                                          deriving (Show, Eq)

type Date = (Int, Month, Int)
type Price = Float
type Quantity = Int

type Supply = (String, Quantity, Price)
-- representing the name of the ingredient,
--  the quantity needed of that ingredient,
--  and the total price of the needed quantity

type Delivery = (Date, [Supply])
-- date of delivery the restaurant will make and the required supply on that date.

data Ingredient =
    SimpleIngredient String  -- A basic ingredient
  | Recipe String [Ingredient] deriving (Show, Eq)
  -- An ingredient consisting of other ingredients

data Expense =
    Item String Price Date
    -- A single expense item
  | Category String [Expense]
  -- A category of expenses that could contain expenses or other categories.
  deriving (Show, Eq)
```

3. Database

You will be provided with a starting **.hs** file that contains the data types mentioned in the previous section, as well as the data you will be using for testing written in the form of Haskell functions. They are the following:

a) `ingredient_info :: [(String, Int, Price)]`

This function returns a list of tuples. Each tuple contains the name of the ingredient, the number of days it takes to deliver it, and the price of the unit.

b) `shopping_list :: [(Date, [Ingredient])]`

This function returns a list of pairs. Each pair contains a date and a list of ingredients needed on that date.

4. Implementation Requirements

Below are the functions you are required to implement along with their type signatures and descriptions. You will need to implement additional functions to break down the logic over them for a more organized structure. You must follow any instructions related to how to implement a specific function. ***Make sure your implementation is WinHugs compatible.***

a) `calculateDeliveryDates`

```
calculateDeliveryDates :: Date -> [Ingredient] -> [(Date, (String, Price))]
```

For a given list of ingredients and the date they are required on, it calculates the deliveries needed. This must handle deliveries across different months (ingredient required in January, so it must be delivered in December, for example).

Example:

Input:

```
calculateDeliveryDates (10, Feb, 2026) [SimpleIngredient "rice",
    SimpleIngredient "apples",
    Recipe "dough" [SimpleIngredient "flour", SimpleIngredient "eggs"],
    SimpleIngredient "apples"
]
```

Output:

```
[
    ((21,Jan, 2026),("rice",1.2)),
    ((5,Feb, 2026),("apples",5.0)),
    ((9,Feb, 2026),("flour",0.5)),
    ((9,Feb, 2026),("eggs",2.0)),
    ((5,Feb, 2026),("apples",5.0))
]
```

In the above example, "apples" take 5 days to arrive, so they will be available on 5 Feb. Flour and eggs take 3 days to arrive, so they will be available on 9 Feb

b) `summarizeAllDeliveries`

```
summarizeAllDeliveries :: [Date] -> [Delivery]
```

Given a range of dates, it looks up the data in `shpping_list` and `ingredient_info` and gives a list of all deliveries that need to be made, grouped by date, where each date occurs once, showing all the ingredients that need to be delivered and the total quantity needed from each. These deliveries should account for delivery time to ensure each ingredient arrives exactly on the day it is needed. Within each date's list, each unique ingredient appears only once showing the total number of units required and the total price of this quantity required. The final return list should be sorted by date and within each date, ingredients should be sorted alphabetically.

Example:

Input:

```
summarizeAllDeliveries [(15, Feb, 2026), (17, Feb, 2026)]
```

Output:

```
[
((26,Jan, 2026),[("rice",1,1.2)]),
((10,Feb, 2026),[("sugar",1,6.0)]),
((14,Feb, 2026),[("butter",1,12.0),("eggs",1,2.0),("flour",1,0.5)]),
((16,Feb, 2026),[("eggs",1,2.0),("flour",3,1.5)])
]
```

c) `getDeliveryExpenses`

```
getDeliveryExpenses :: [Delivery] -> Expense
```

Given a list of deliveries, the function returns the Expense for these deliveries. The expense will be in the "Food Supplies" category, and it contains a list of expenses where each ingredient is an item.

Example:

Input:

```
getDeliveryExpenses [
((26,Jan, 2026),[("rice",1,1.2)]),
((10,Feb, 2026),[("sugar",1,6.0)]),
((14,Feb, 2026),[("butter",1,12.0),("eggs",1,2.0),("flour",1,0.5)]),
((16,Feb, 2026),[("eggs",1,2.0),("flour",3,1.5)])
]
```

Output:

```
Category "Food Supplies" [
            Item "rice" 1.2 (26,Jan, 2026),
            Item "sugar" 6.0 (10,Feb, 2026),
            Item "butter" 12.0 (14,Feb, 2026),
            Item "eggs" 2.0 (14,Feb, 2026),
```

```
                Item "flour" 0.5 (14,Feb, 2026),
                Item "eggs" 2.0 (16,Feb, 2026),
                Item "flour" 1.5 (16,Feb, 2026)
                ]
```

d) `mostPopularDish`

`mostPopularDish :: [String] -> [String]`

Given a list of dish names ordered by customers, it returns a list of the most frequently ordered items. In case of a tie, return all of the most popular dishes. In case of an empty list, return an empty list.

Example:

Input:

```
mostPopularDish ["rice", "soup", "cake", "rice"]
```

Output:

```
["rice"]
```

e) `calculateTotalExpenses`

`calculateTotalExpenses :: Expense -> Price`

Given a tree of expenses, this function returns the total value of expenses. Must be implemented using higher-order functions

Example:

Input:

```
calculateTotalExpenses   (Category "Restaurant Expenses" [
        Category "Food Supplies" [
            Item "Vegetables" 450.50 (12, Jan, 2026),
            Item "Meat" 890.00 (13, Jan, 2026)
        ],
        Category "Salaries" [
            Item "Chef" 3500.00 (1, Jan, 2026),
            Item "Waiter" 2000.00 (1, Jan, 2026)
        ]
    ])
```

Output: `6840.5`

f) `countCategoryItems`

`countCategoryItems :: String -> Expense -> Int`

Given a category name and a tree of expenses, this function returns the number of items under this category (number of leaves in that subtree).

Example:

Input:

```
countCategoryItems  "Salaries" (Category "Restaurant Expenses" [
        Category "Food Supplies" [
            Item "Vegetables" 450.50 (12, Jan, 2026),
            Item "Meat" 890.00 (13, Jan, 2026)
        ],
        Category "Salaries" [
        (Category "Kitchen" [
            Item "Chef" 3500.00 (1, Jan, 2026)]),
        (Category "Service"
            [Item "Waiter" 2000.00 (1, Jan, 2026),
            Item "Waiter" 2100.00 (3, Jan, 2026)])
        ]
    ])
```

Output: `3`